

Aufgabe 02: Kommunikation in verteilten Systemen mit Remote Procedure Calls (RPC)

Internetprotokolle/Netze 2

Abgabedatum: 24. April 2026

Abnahmedatum: 27. April 2026

Lernziele

- Verständnis der Grundlagen von Remote Procedure Calls (RPC)
- Entwicklung eines eigenen RPC-Systems
- Prinzipien von RESTful Web Services
- Modellierung der Kommunikation in verteilten Systemen

1 Einführung

In der Vorlesung wurde das Konzept des Remote Procedure Call (RPC) mit gRPC als Beispiel behandelt. In dieser Übung werden wir uns mit dem JSON-RPC-Protokoll beschäftigen, das eine einfache und leichtgewichtige Methode zur RPC Implementierung bietet. JSON-RPC ist ein Protokoll, das es ermöglicht, Remote Procedure Calls über das JSON-Format zu tätigen.

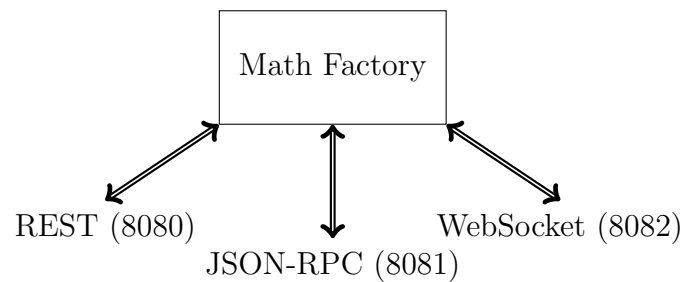
Information zu JSON-RPC finden Sie in der offiziellen Spezifikation: <https://www.jsonrpc.org/specification>.

Recherchieren Sie, was JSON-RPC ist und wie es funktioniert. Achten Sie dabei besonders auf die Aspekte, die in der Vorlesung behandelt wurden, z.B. Serialisierung, Transportprotokolle, Rückwärtskompatibilität, etc.

2 Aufgabe

Im Rahmen dieser Übung ist die Entwicklung eines JSON-RPC Clients und Servers vorgesehen. Der sogenannte „Math Factory“ Server soll, analog zur ersten Übung, Rechenoperationen im Auftrag der Clients ausführen. Über JSON-RPC (Port Nr.: 8081) sollen Clients die Möglichkeit erhalten, diese Operationen remote aufzurufen.

Zusätzlich soll der Server eine REST Schnittstelle (Port Nr.: 8080) bereitstellen, um die unterstützten Rechenoperationen zu verwalten. Darüber hinaus ist die Implementierung einer WebSocket Schnittstelle (Port Nr.: 8082) erforderlich, über die Clients benachrichtigt werden, wenn sie eine übermäßige Anzahl von Anfragen an den Server senden.



Q1: [2 points] Vergleich von JSON-RPC und gRPC: JSON-RPC verwendet viele der in der Vorlesung behandelten Bausteine, wie beispielsweise JSON-Serialisierung und HTTP-Transport, um eine RPC-Schnittstelle bereitzustellen. Vergleichen Sie JSON-RPC mit gRPC insbesondere in Bezug auf die genannten Aspekte. Analysieren Sie die Vor- und Nachteile der Verwendung von JSON-RPC im Vergleich zu gRPC und geben Sie Empfehlungen für Anwendungsfälle, in denen JSON-RPC besonders geeignet ist. Insbesondere sollten Sie die folgenden Aspekte berücksichtigen:

- Serialisierung
- Transportprotokolle
- Rückwärtskompatibilität
- Performance
- Angebotene Fehlersemantiken (z.B. At-Most-Once, At-Least-Once, Exactly-Once)

Q2: [5 points] Implementierung eines HTTP-Servers: Entwickeln Sie einen Server, der die folgenden Rechenoperationen unterstützt:

Operation	Beschreibung	Kosten (Rechenzeit)
Addition	$a + b$	2
Subtraktion	$a - b$	3
Multiplikation	$a * b$	25
Division	a / b	50
Factorial	$a!$	100
Potenzierung	a^b	1150

Tabelle 1: Unterstützte Rechenoperationen und ihre Kosten

Die Funktionen sind als Ressourcen innerhalb einer RESTful API zu verwalten. Über die REST-Schnittstelle soll es Administratoren ermöglicht werden, eine Serverinstanz zu konfigurieren, ohne dass sie den Server neu starten müssen, indem sie festlegen, welche Rechenoperationen verfügbar und ausführbar sind sowie welche Kosten diese verursachen. Modellieren Sie die vom Server verwalteten Daten und deren entsprechende Serialisierungen. Verwenden Sie OpenAPI zur Dokumentation und zum Testen der API.

Zur Implementierung kann eine geeignete Programmiersprache wie beispielsweise Python in Kombination mit der Bibliothek Flask verwendet werden. Alternativ steht es Ihnen frei, eine andere Programmiersprache Ihrer Wahl einzusetzen. Gerne können Sie auch Ihre Implementierung aus der ersten Übung erweitern, um die Anforderungen dieser Aufgabe zu erfüllen.

Q3: [5 Punkte] Implementierung von JSON-RPC: Entwickeln Sie JSON-RPC-Methoden für den Server, die es ermöglichen, verschiedene Rechenoperationen auszuführen. Der Server soll die Anfragen der Clients verarbeiten und die entsprechenden Ergebnisse zurückliefern.

Verwenden Sie ein Transportprotokoll Ihrer Wahl und stellen Sie sicher, dass die JSON-RPC-Spezifikation eingehalten wird. Ungültige Anfragen (z. B. nicht verfügbare Funktionen auf dem Server) sollen mit einem entsprechenden Fehlercode beantwortet werden.

Q4: [3 Punkte] Implementierung eines JSON-RPC Clients: Entwickeln Sie eine Client-Anwendung, die eine unendliche Reihe mithilfe von JSON-RPC berechnet. Als Beispiel können Sie die Funktion $f(x) = e^x$ verwenden, die durch die folgende Taylor-Reihe dargestellt wird:

$$f(x) = e^x = \sum_{n=0}^N \frac{x^n}{n!}$$

Der Client soll die Berechnung der Funktion an den Server delegieren, die Ergebnisse empfangen und diese schrittweise anzeigen. Der N kann hier als eine Input-Variable betrachtet werden. Für diese Aufgabe, konfigurieren wir den Server so, dass alle Rechenoperationen verfügbar sind und die Kosten wie in Tabelle 1 angegeben sind.

Q5: [3 Punkte] Implementierung der WebSocket-Schnittstelle: Erweitern Sie die Implementierung von Server und Client so, dass der Client seine Berechnung am Anfang mit einer UUID startet und diese in jeder seiner Remote Aufruf mitschickt. Der Server soll eine Abrechnung der Anfragen durchführen und den Client über WebSocket Schnittstelle benachrichtigen, wenn die verursachten Kosten einen definierten Schwellenwert überschreiten. Die Schwelle soll auch vom Client über die WebSocket Schnittstelle gesetzt bzw. geändert werden können.

Passen Sie die Implementierung des Clients und Server entsprechend an, um die WebSocket-Schnittstelle zu nutzen. Zeigen Sie die Funktionalität der WebSocket-Schnittstelle, indem Sie den Client so konfigurieren, dass er während seiner Berechnung von dem Server benachrichtigt wird, woraufhin er seine Berechnung anhält und die bisherigen Ergebnisse anzeigt.

Q6: [2 Punkte] Containerisierung: Containerisieren Sie Ihren Server und Client mit Docker. Erstellen Sie Dockerfiles für beide Komponenten und verwenden Sie Docker Compose, um die gesamte Anwendung zu orchestrieren. Stellen Sie sicher, dass die Container korrekt miteinander kommunizieren können und dass die Ports entsprechend weitergeleitet werden.

Hinweis: In der Regel sind die APIs immer zu schützen mit Authentifizierungs- und Autorisierungsmechanismen. Solche Mechanismen sind in dieser Übung nicht berücksichtigt.